6.1

Álgebra linear e cálculo mínimos para ML

Vetores, matrizes, produto interno, norma, derivada parcial e gradiente — só o necessário, todo em NumPy vetorizado, sem loops.

PRÁTICA
Reescrever loops como operações matriciais

AMBIENTE
Google Colab · NumPy

DURAÇÃO
2 horas

06

Machine Learning na mão, com NumPy

Nenhum import de sklearn. Cada algoritmo é derivado no quadro, escrito em NumPy e só então comparado com a versão da biblioteca, que passa a ser óbvia.

6.1	Álgebra linear e cálculo
6.2	Equação normal
6.3	Gradiente descendente
6.4	Regressão logística
6.5	Métricas na mão
6.6	Validação cruzada
6.7	KNN
6.8	Naive Bayes
6.9	Árvore de decisão
6.10	K-Means
6.11	PCA
6.12	Perceptron e MLP
6.13	Laboratório 2

13 aulas · termina com a sua mini-biblioteca no GitHub

Ao final da aula, você vai conseguir

1. Representar amostras, datasets e pesos como arrays e prever o shape de qualquer operação entre eles.
2. Calcular produto interno, norma L1, norma L2 e distância entre vetores, à mão e em NumPy.
3. Calcular as derivadas parciais e o gradiente de uma função de duas variáveis.
4. Escrever o gradiente do erro quadrático médio em uma linha matricial e conferi-lo numericamente.
5. Reescrever um laço sobre amostras como uma operação vetorizada.
6. Interpretar o produto interno geometricamente: ângulo, cosseno e projeção.

Aprofundamento (seção A): Hessiana, convexidade, autovalores e número de condição.

Do módulo 5 para o módulo 6

O QUE FICOU DO MÓDULO 5

- 5.1 O ciclo: carregar, separar, treinar, prever, avaliar
- 5.2 O teste fica trancado até o fim
- 5.3 Pipeline e ColumnTransformer
- 5.4 Escalonamento e codificação
- 5.5 Seleção de features
- 5.6 Titanic de ponta a ponta

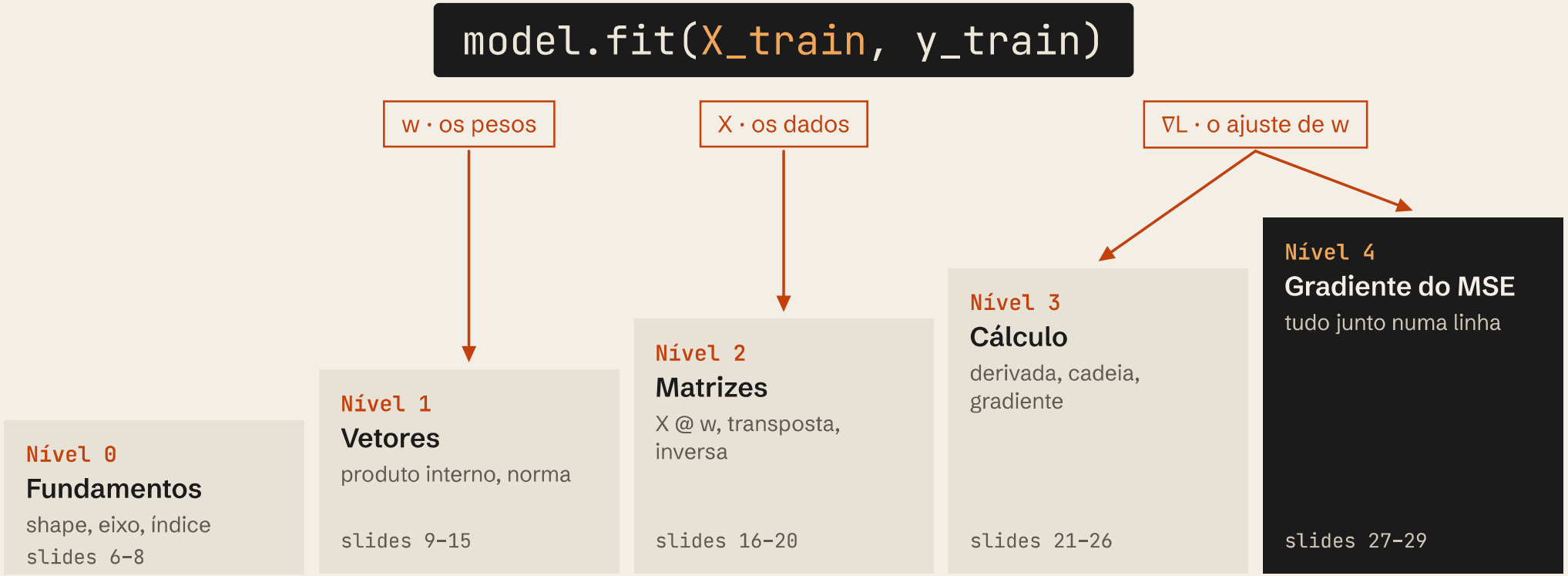
O GANCHO DE HOJE

```
model.fit(X_train, y_train)
```

Em todo o módulo 5, o treino coube nessa linha. Dentro dela existem uma matriz $\mathbf{X}$, um vetor de pesos $\mathbf{w}$ e um gradiente que diz como ajustar $\mathbf{w}$.

Hoje aprendemos a ler essas três peças. Nas próximas aulas, a escrevê-las.

Mapa da aula


model.fit(X_train, y_train)
w · os pesos
X · os dados
∇L · o ajuste de w
Nível 0
Fundamentos
shape, eixo, índice
slides 6-8
Nível 1
Vetores
produto interno, norma
slides 9-15
Nível 2
Matrizes
X @ w, transposta, inversa
slides 16-20
Nível 3
Cálculo
derivada, cadeia, gradiente
slides 21-26
Nível 4
Gradiente do MSE
tudo junto numa linha
slides 27-29

Escalar, vetor, matriz, tensor

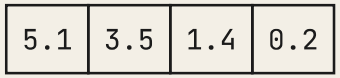
NÍVEL 0 · FUNDAMENTOS



Escalar

3.5

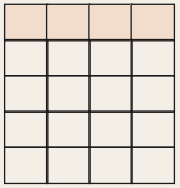
`ndim` 0
`shape` ()



Vetor

uma flor do Iris

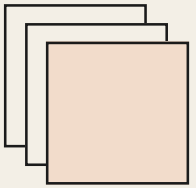
`ndim` 1
`shape` (4,)



Matriz

o Iris inteiro

`ndim` 2
`shape` (150, 4)



Tensor

32 imagens 28×28

`ndim` 3
`shape` (32, 28, 28)

```
np.array(3.5).ndim              # 0
x = np.array([5.1, 3.5, 1.4, 0.2])
x.ndim, x.shape                 # (1, (4,))
imgs = np.zeros((32, 28, 28))
imgs.ndim, imgs.shape           # (3, (32, 28, 28))
```

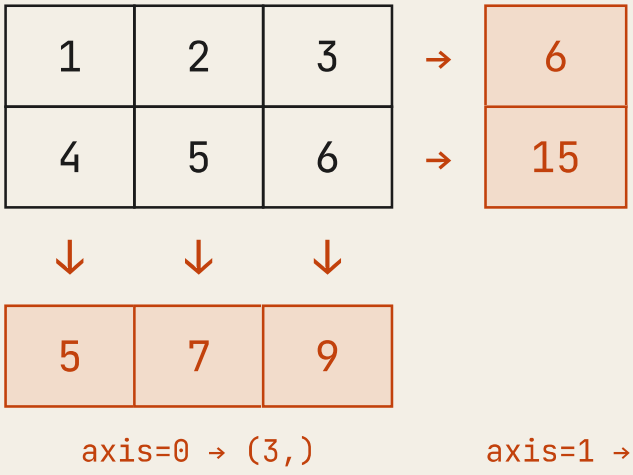
`ndim`: quantos eixos.

`shape`: o tamanho de cada eixo.

Antes de qualquer operação, confira o shape.

Eixos e agregações

NÍVEL 0 · FUNDAMENTOS




```
M = np.array([[1, 2, 3], [4, 5, 6]])  # (2, 3)
M.sum(axis=0)   # [5, 7, 9]  (3,)
M.sum(axis=1)   # [6, 15]    (2,)
```

O axis que você passa é o que desaparece do shape.

axis=0 colapsa as linhas: um valor por coluna (feature). axis=1 colapsa as colunas: um valor por linha (amostra). Vale para `sum`, `mean`, `max` e `std`.

Indexar, fatiar, filtrar e mudar o shape

NÍVEL 0 · FUNDAMENTOS

IRIS · X (150, 4) · Y (150,)

`X[0]`	`(4,)`	a primeira flor
`X[:, 2]`	`(150,)`	a terceira feature
`X[:10]`	`(10, 4)`	as dez primeiras flores
`X[X[:, 0] > 5]`	`(k, 4)`	máscara booleana: sépala > 5 cm
`y.reshape(-1, 1)`	`(150, 1)`	y como coluna
`x[:, None]`	`(4, 1)`	x = X[0] como coluna

FATIAS SÃO VIEWS

Alterar a fatia altera o original. Use .copy() quando for modificar.

```
M = np.arange(6).reshape(2, 3)
r = M[0]          # view
r[0] = 99
M[0, 0]           # 99
r = M[0].copy()   # cópia
```

Vetor: uma amostra como lista de números

NÍVEL 1 · VETORES

Cada flor do Iris tem quatro medidas. Para o modelo, ela é um ponto num espaço de quatro dimensões.

$\mathbf{x} =$

5.1	3.5	1.4	0.2
comprimento da sépala	largura da sépala	comprimento da pétala	largura da pétala

```
x = np.array([5.1, 3.5, 1.4, 0.2])
x.shape   # (4,)
x[2]      # 1.4
```

Notação: x_i é a i-ésima componente; d é o número de componentes.

O shape `(4,)` é um array 1D. Não é linha nem coluna.

Vetor como seta

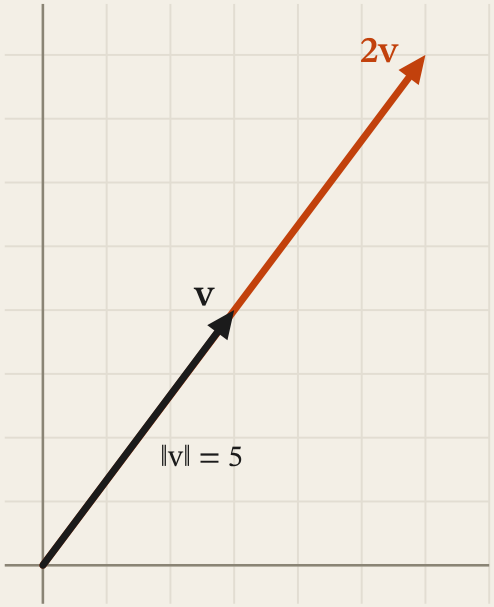
NÍVEL 1 · VETORES

$$\mathbf{v} = [3, 4]$$

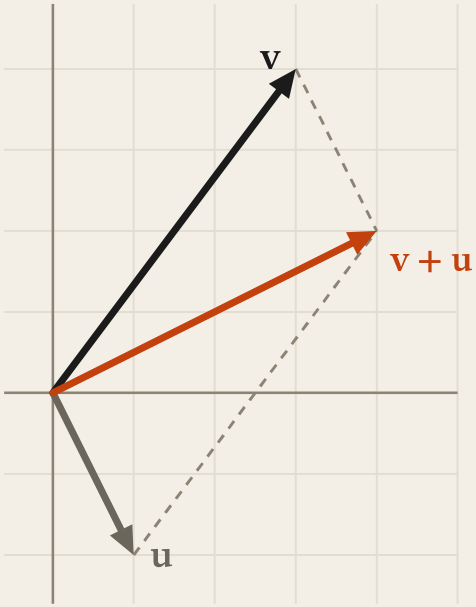
$$\|\mathbf{v}\| = \sqrt{(9 + 16)} = 5$$

2 · v = [6, 8]
mesma direção, dobro do comprimento

[3, 4] + [1, –2] = [4, 2]
regra do paralelogramo



ESCALA



SOMA

Operações elemento a elemento e broadcasting

NÍVEL 1 · VETORES

MESMO SHAPE: POSIÇÃO POR POSIÇÃO

```
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])

a + b   # [11, 22, 33]
a * b   # [10, 40, 90]
2 * a   # [2, 4, 6]
a ** 2  # [1, 4, 9]
```

Nenhum laço: o NumPy percorre os elementos em C.

SHAPES DIFERENTES: BROADCASTING

X	(150, 4)
mu	(4,)
X - mu	(150, 4)

Compare os shapes da direita para a esquerda. Cada par precisa ser igual, ou um dos dois valer 1, ou faltar.

`X - X.mean(axis=0)` centraliza as 150 flores de uma vez.

Produto interno

NÍVEL 1 · VETORES

$$\mathbf{x} \cdot \mathbf{w} = \sum_i x_i w_i = x_1 w_1 + x_2 w_2 + \ldots + x_d w_d$$

NO QUADRO

x	[1, 2, 3]
w	[0.5, -1, 2]
x * w	[0.5, -2, 6]
soma	4.5

NO NUMPY

```
x = np.array([1, 2, 3])
w = np.array([0.5, -1, 2])
x @ w           # 4.5
np.dot(x, w)    # 4.5
np.sum(x * w)   # 4.5
```

É a previsão de um modelo linear: cada feature pesada pelo seu coeficiente. Volta nas aulas 6.2 e 6.4.

Produto interno com geometria

NÍVEL 1 · VETORES

$$\mathbf{x} \cdot \mathbf{w} = \|\mathbf{x}\| \, \|\mathbf{w}\| \cos\theta$$

EXEMPLO

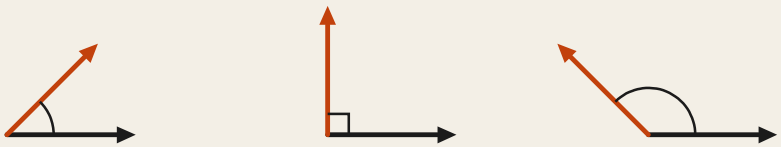
$\mathbf{a} = [1, 0], \mathbf{b} = [1, 1]$

$\mathbf{a} \cdot \mathbf{b} = 1$

$\|\mathbf{a}\| = 1, \|\mathbf{b}\| = \sqrt{2}$

$\cos\theta = 1/\sqrt{2} \approx 0.707 \rightarrow \theta = 45°$

```
a = np.array([1., 0.]); b = np.array([1., 1.])
cos_t = a @ b / (np.linalg.norm(a) * np.linalg.norm(b))
np.degrees(np.arccos(cos_t))   # ≈ 45.0
```



agudo
`a · b > 0`

90°, ortogonais
`a · b = 0`

obtuso
`a · b < 0`

Similaridade de cosseno e projeção

NÍVEL 1 · VETORES

SIMILARIDADE DE COSSENO

$$\cos(\mathbf{a}, \mathbf{b}) = \mathbf{a}\cdot\mathbf{b} \,/\, (\|\mathbf{a}\|\,\|\mathbf{b}\|)$$

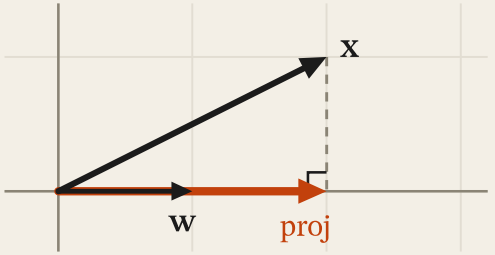
`[1, 2]` e `[2, 4]`: 10 / (√5 · √20) = **1**. Mesma direção, tamanhos diferentes.

```
a = np.array([1, 2]); b = np.array([2, 4])
cos = a @ b / (np.linalg.norm(a) * np.linalg.norm(b))
cos   # 1.0
```

PROJEÇÃO

$$\text{proj}_{\mathbf{w}}(\mathbf{x}) = (\mathbf{x}\cdot\mathbf{w} \,/\, \mathbf{w}\cdot\mathbf{w})\,\mathbf{w}$$

$\mathbf{x}$ = [2, 1], $\mathbf{w}$ = [1, 0] → (2 / 1)·[1, 0] = **[2, 0]**



A regressão linear (6.2) é a projeção de $\mathbf{y}$ sobre o espaço das colunas de $\mathbf{X}$.

Norma e distância

NÍVEL 1 · VETORES

NORMA L2

$$\|\mathbf{x}\|_2 = \sqrt{(\Sigma\, x_i^2)}$$

`[3, 4]` → 5

`np.linalg.norm(x)`

NORMA L1

$$\|\mathbf{x}\|_1 = \Sigma\, |x_i|$$

`[3, 4]` → 7

`np.linalg.norm(x, ord=1)`

NORMA L∞

$$\|\mathbf{x}\|\infty = \max |x_i|$$

`[3, 4]` → 4

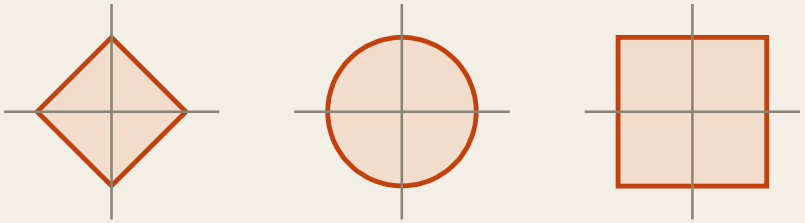
`np.linalg.norm(x, ord=np.inf)`

DISTÂNCIA

$$d(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|$$

`[1,1], [4,5]` → 5

`np.linalg.norm(a - b)`



BOLAS UNITÁRIAS: ‖X‖ = 1

O formato do losango explica por que Lasso zera pesos (aula 7.1).

Onde volta: KNN 6.7 · K-Means 6.10 · Ridge e Lasso 7.1

Matriz: o dataset inteiro

NÍVEL 2 · MATRIZES

$$\mathbf{X} = \begin{bmatrix} 5.1 & 3.5 & 1.4 & 0.2 \\ 4.9 & 3.0 & 1.4 & 0.2 \\ 4.7 & 3.2 & 1.3 & 0.2 \\ 4.6 & 3.1 & 1.5 & 0.2 \\ \vdots & \vdots & \vdots & \vdots \end{bmatrix}$$

150 linhas (amostras) × 4 colunas (features)

`X.shape`	(n, d) = (150, 4)
`X[0]`	a primeira flor, shape (4,)
`X[:, 2]`	a terceira feature, shape (150,)
`X.T`	a transposta $\mathbf{X}^{\mathrm{T}}$, shape (4, 150)

Convenção do módulo: n amostras nas linhas, d features nas colunas. É o mesmo formato que o sklearn espera.

Produto matricial e a regra dos shapes

NÍVEL 2 · MATRIZES

As dimensões de dentro precisam ser iguais. As de fora formam o shape do resultado.

`X @ w`	`(n, d)`	`@ (d,)`	→	`(n,)`	as n previsões de uma vez
`X @ W`	`(n, d)`	`@ (d, k)`	→	`(n, k)`	k saídas por amostra
`X.T @ X`	`(d, n)`	`@ (n, d)`	→	`(d, d)`	aparece na aula 6.2
`X @ X`	`(n, d)`	`@ (n, d)`	→	`erro`	d ≠ n: não multiplica

Cada elemento do resultado é um produto interno: linha i da esquerda com coluna j da direita.

Duas leituras de X @ w

NÍVEL 2 · MATRIZES

$$\mathbf{X}\begin{bmatrix}1 & 1\\ 1 & 2\\ 1 & 3\end{bmatrix}\mathbf{w}\begin{bmatrix}0.5\\ 1\end{bmatrix}=\begin{bmatrix}1.5\\ 2.5\\ 3.5\end{bmatrix}$$

`(3, 2) @ (2,) → (3,)`

LEITURA 1 · LINHAS

```
[1, 1] · [0.5, 1] = 1.5
[1, 2] · [0.5, 1] = 2.5
[1, 3] · [0.5, 1] = 3.5
```

Cada saída é o produto interno de uma linha de X com w.

LEITURA 2 · COLUNAS

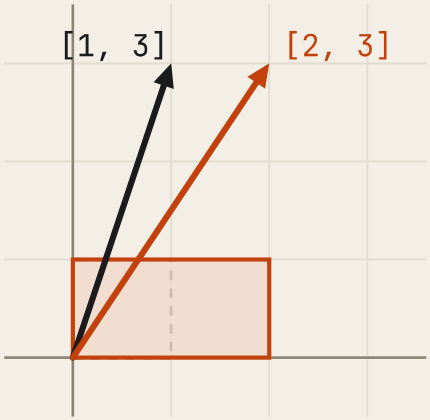
$$0.5\times\begin{bmatrix}1\\ 1\\ 1\end{bmatrix}+1\times\begin{bmatrix}1\\ 2\\ 3\end{bmatrix}=\begin{bmatrix}1.5\\ 2.5\\ 3.5\end{bmatrix}$$

$\mathbf{Xw} = w_1\cdot(\text{coluna 1}) + \ldots + w_d\cdot(\text{coluna } d)$: uma combinação das features.

Matriz como transformação

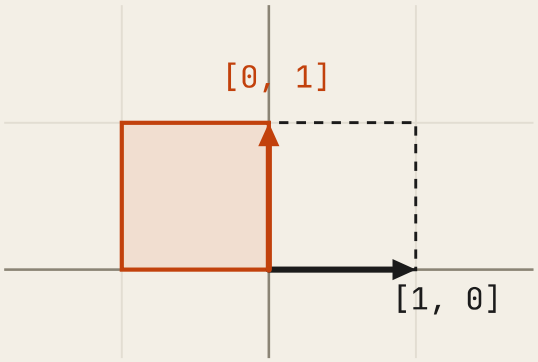
NÍVEL 2 · MATRIZES

`A = [[2, 0], [0, 1]]`



estica o eixo x · `A @ [1, 3] = [2, 3]`

`R = [[0, -1], [1, 0]]`



gira 90° · `R @ [1, 0] = [0, 1]`

Multiplicar por uma matriz = transformar o espaço.

As colunas da matriz dizem para onde vão [1, 0] e [0, 1].

```
A = np.array([[2, 0], [0, 1]])
A @ np.array([1, 3])   # [2, 3]
R = np.array([[0, -1], [1, 0]])
R @ np.array([1, 0])   # [0, 1]
```

Transposta, identidade e inversa

NÍVEL 2 · MATRIZES

TRANSPOSTA

$$(\mathbf{AB})^{\mathrm{T}} = \mathbf{B}^{\mathrm{T}}\mathbf{A}^{\mathrm{T}}$$

`A (n, d) · B (d, k) → AB (n, k)`
`(AB)ᵀ (k, n) = Bᵀ (k, d) · Aᵀ (d, n)`

IDENTIDADE

$$\mathbf{A}\,\mathbf{I} = \mathbf{I}\,\mathbf{A} = \mathbf{A}$$

```
I = np.eye(d)
```

INVERSA

$$\mathbf{A}\,\mathbf{A}^{-1} = \mathbf{I}$$

só para matriz quadrada com det ≠ 0

QUANDO NÃO EXISTE INVERSA

$$\begin{vmatrix} 1 & 2 \\ 2 & 4 \end{vmatrix} \quad \det = 1\cdot 4 - 2\cdot 2 = 0$$

A segunda coluna é o dobro da primeira. Features colineares deixam $X^{\mathrm{T}}X$ singular: problema na 6.2 e motivo do Ridge (7.1).

```
S = np.array([[1., 2.], [2., 4.]])
np.linalg.det(S)        # 0.0: singular
M = np.array([[2., 1.], [1., 3.]])
b = np.array([3., 5.])
np.linalg.solve(M, b)   # [0.8, 1.4], não inv(M) @ b
```

Derivada como limite

NÍVEL 3 · CÁLCULO

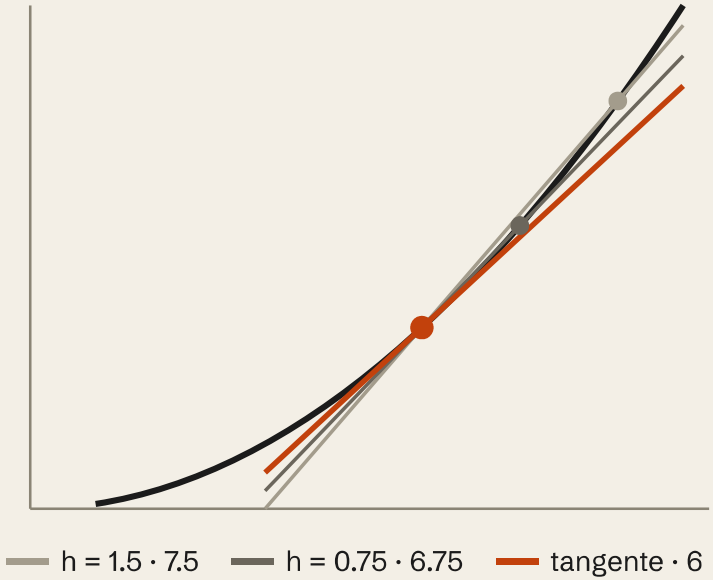
$$f'(x) = \lim_{h \to 0} [f(x+h) - f(x)] / h$$

$f(x) = x^2$, em $x = 3$, com $h = 0.1$:

PROGRESSIVA $(3.1^2 - 9) / 0.1 = 6.1$ erro de ordem h

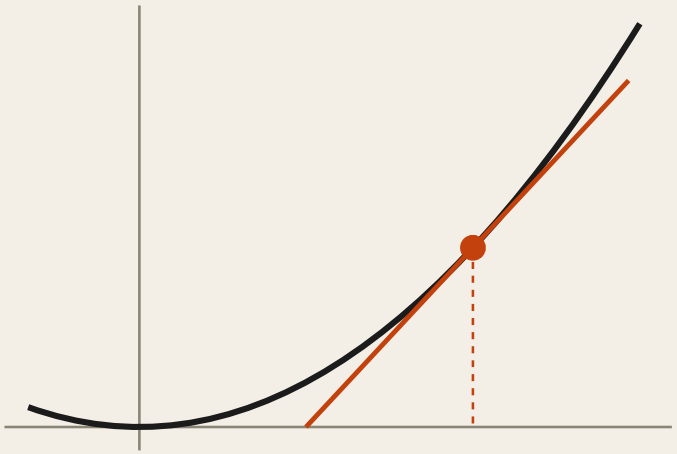
CENTRAL $(3.1^2 - 2.9^2) / 0.2 = 6.0$ erro de ordem h^2

```
f = lambda x: x ** 2
h = 0.1
(f(3 + h) - f(3)) / h             # ≈ 6.1
(f(3 + h) - f(3 - h)) / (2 * h)   # ≈ 6.0
```



Derivada: a inclinação num ponto

NÍVEL 3 · CÁLCULO



$f(x) = x^2$ e a tangente em $x = 3$, inclinação 6

$$f'(x) \approx [f(x+h) - f(x-h)] \,/\, 2h$$

POTÊNCIA $(x^n)' = n\,x^{n-1}$

CADEIA $f(g(x))' = f'(g(x)) \cdot g'(x)$

EXEMPLO $((2x+1)^2)' = 2(2x+1) \cdot 2$

```
f = lambda x: x ** 2
h = 1e-5
(f(3 + h) - f(3 - h)) / (2 * h)   # ≈ 6.0
```

Regras que vamos usar no módulo

NÍVEL 3 · CÁLCULO

Constante	$(c)' = 0$	$(7)' = 0$
Soma	$(f+g)' = f' + g'$	$(x^2 + x)' = 2x + 1$
Potência	$(x^n)' = nx^{n-1}$	$(x^3)' = 3x^2$
Produto	$(fg)' = f'g + fg'$	$(xe^x)' = e^x + xe^x$
Cadeia	$f(g(x))' = f'(g(x)) \cdot g'(x)$	$(e^{2x})' = 2e^{2x}$
Exponencial	$(e^x)' = e^x$	
Logaritmo	$(\ln x)' = 1/x$	$(\ln 2x)' = 1/x$

DESTAQUE · AULA 6.4

$$\sigma(z) = 1 / (1 + e^{-z})$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Em $z = 0$: $\sigma = 0.5$ e $\sigma' = 0.25$, a maior inclinação da curva.

Regra da cadeia passo a passo

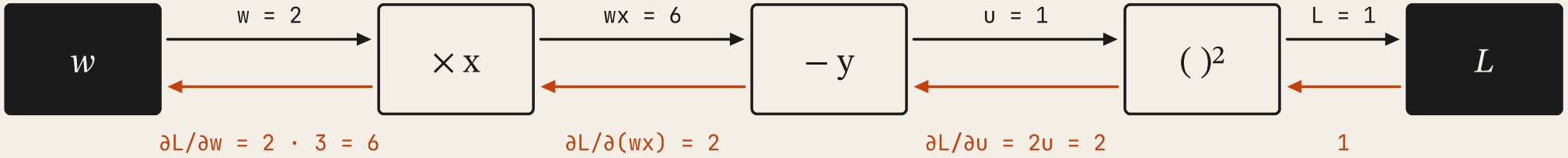
NÍVEL 3 · CÁLCULO

$L(w) = (w\,x - y)^2$

com $u = w\,x - y$: $\mathrm{d}L/\mathrm{d}w = 2u \cdot x$

COM NÚMEROS

$w = 2, x = 3, y = 5 \rightarrow u = 1 \rightarrow \mathrm{d}L/\mathrm{d}w = 2 \cdot 1 \cdot 3 = 6$



Ida em preto: valores. Volta em laranja: derivadas locais multiplicadas. Prévia do backpropagation da aula 6.12.

Derivada parcial e gradiente

NÍVEL 3 · CÁLCULO

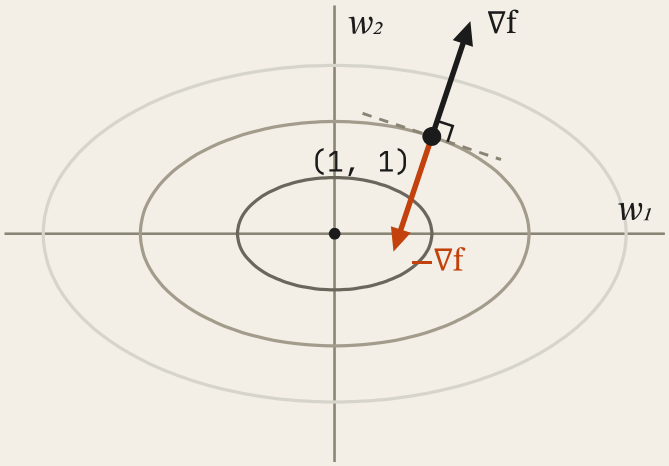
$$f(w_1, w_2) = w_1^2 + 3w_2^2$$

$$\partial f / \partial w_1 = 2w_1$$

$$\partial f / \partial w_2 = 6w_2$$

$\nabla f = [2w_1, 6w_2] \rightarrow$ em $(1, 1)$: $[2, 6]$

Uma parcial deriva em relação a uma variável e trata as outras como constantes. O gradiente junta todas as parciais num vetor do mesmo shape que $\mathbf{w}$.



∇f sobe mais rápido, perpendicular à curva

$-\nabla f$ desce

Para onde o gradiente aponta

NÍVEL 3 · CÁLCULO

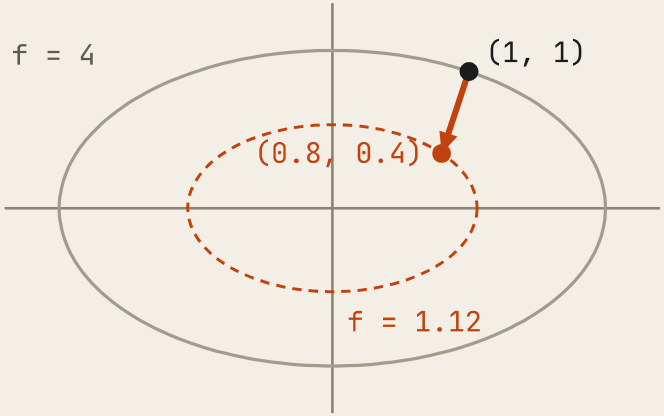
$$D_u f = \nabla f \cdot \mathbf{u} = \|\nabla f\| \cos\theta$$

Máxima quando $\mathbf{u}$ aponta na direção de ∇f ($\theta = 0$). Para descer, ande contra o gradiente:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla f$$

w = (1, 1)	f = 4
∇f	[2, 6]
w - 0.1·[2, 6]	[0.8, 0.4]
f novo	0.64 + 0.48 = 1.12

ISSO VIRA UM ALGORITMO NA AULA 6.3



Gradiente do erro quadrático médio

NÍVEL 4 · GRADIENTE DO MSE

$$\mathbf{e} = \mathbf{X}\,\mathbf{w} - \mathbf{y}$$

$$L(\mathbf{w}) = (1/n)\,\Sigma_i\, e_i^2$$

$$\partial L \,/\, \partial w_j = (2/n)\,\Sigma_i\, e_i\, X_{ij}$$

$$\nabla L = (2/n)\,\mathbf{X}^{\mathrm{T}}(\mathbf{X}\,\mathbf{w} - \mathbf{y})$$

`X.T`	`(d, n)`
`X @ w - y`	`(n,)`
`gradiente`	`(d,)`

```
grad = 2 / n * X.T @ (X @ w - y)
```

Igualar a zero e resolver em $\mathbf{w}$: aula 6.2

Andar contra ele, passo a passo: aula 6.3

Derivação matricial no próximo slide →

Derivando ∇L na forma matricial

NÍVEL 4 · GRADIENTE DO MSE

DEFINIÇÃO	$L(\mathbf{w}) = (1/n)(\mathbf{X}\mathbf{w} - \mathbf{y})^{\mathrm{T}}(\mathbf{X}\mathbf{w} - \mathbf{y})$	$\lVert\mathbf{e}\rVert^2 = \mathbf{e}^{\mathrm{T}}\mathbf{e}$
EXPANDIR	$= (1/n)(\mathbf{w}^{\mathrm{T}}\mathbf{X}^{\mathrm{T}}\mathbf{X}\mathbf{w} - 2\mathbf{y}^{\mathrm{T}}\mathbf{X}\mathbf{w} + \mathbf{y}^{\mathrm{T}}\mathbf{y})$	$\mathbf{y}^{\mathrm{T}}\mathbf{X}\mathbf{w} = \mathbf{w}^{\mathrm{T}}\mathbf{X}^{\mathrm{T}}\mathbf{y}$ é um número
IDENTIDADES	$\nabla(\mathbf{a}^{\mathrm{T}}\mathbf{w}) = \mathbf{a} \cdot \nabla(\mathbf{w}^{\mathrm{T}}\mathbf{A}\mathbf{w}) = 2\mathbf{A}\mathbf{w}$	$\mathbf{A}$ simétrica; aqui $\mathbf{a} = \mathbf{X}^{\mathrm{T}}\mathbf{y}$, $\mathbf{A} = \mathbf{X}^{\mathrm{T}}\mathbf{X}$
DERIVAR	$\nabla L = (1/n)(2\mathbf{X}^{\mathrm{T}}\mathbf{X}\mathbf{w} - 2\mathbf{X}^{\mathrm{T}}\mathbf{y})$	$\mathbf{y}^{\mathrm{T}}\mathbf{y}$ não depende de $\mathbf{w}$
FATORAR	$\nabla L = (2/n)\,\mathbf{X}^{\mathrm{T}}(\mathbf{X}\mathbf{w} - \mathbf{y})$	o mesmo resultado da soma

Conta pequena à mão

NÍVEL 4 · GRADIENTE DO MSE

$$\mathbf{X} = \begin{vmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{vmatrix} \quad \mathbf{y} = [1, 2, 2] \quad \mathbf{w} = [0, 0]$$

$$\mathbf{e} = \mathbf{X}\mathbf{w} - \mathbf{y} = [-1, -2, -2]$$

$$\mathbf{X}^{\mathrm{T}}\mathbf{e} = [-1-2-2, -1-4-6] = [-5, -11]$$

$$\nabla L = (2/3)\cdot[-5, -11] \approx [-3.33, -7.33]$$

MESMA CONTA EM NUMPY

```
X = np.array([[1, 1], [1, 2], [1, 3]])  # (3, 2)
y = np.array([1, 2, 2])                  # (3,)
w = np.zeros(2)                          # (2,)
e = X @ w - y          # [-1., -2., -2.]
X.T @ e                # [-5., -11.]
2 / 3 * X.T @ e        # ≈ [-3.33, -7.33]
```

Cinco perguntas antes do código

1	X tem shape (200, 3) e `w` tem shape (3,). Qual é o shape de `X @ w`?	A `(3,)`	B `(200,)`	C `(200, 3)`	D `erro`
2	Qual é a norma L2 do vetor `[6, 8]`?	A `14`	B `48`	C `10`	D `100`
3	Para $f(w_1, w_2) = w_1^2 + w_1 w_2$, quanto vale ∇f no ponto (1, 2)?	A `[4, 1]`	B `[2, 1]`	C `[4, 2]`	D `[3, 1]`
4	Qual é o cosseno entre `a = [1, 2]` e `b = [-2, 1]`?	A `1`	B `0`	C `-1`	D `0.5`
5	Se X tem shape (n, d), qual é o shape de $\mathbf{X}^\top\mathbf{X}$?	A `(n, n)`	B `(d, d)`	C `(d,)`	D `(n, d)`

Respostas

1 · B `(200, 3) @ (3,) → (200,)` O 3 de dentro some; sobra uma previsão por amostra.

2 · C $\sqrt{(36 + 64)} = \sqrt{100} = 10$ 14 seria a norma L1.

3 · A $[2w_1 + w_2, w_1] = [4, 1]$ Em relação a w_1, o termo $w_1 w_2$ deriva para w_2.

4 · B `-2 + 2 = 0 → cos = 0` Produto interno zero: os vetores são ortogonais.

5 · B `(d, n) @ (n, d) → (d, d)` Os n de dentro somem; é a matriz da equação normal.

Reescrever loops como operações matriciais

CÉLULA DE DADOS

```
import numpy as np

rng = np.random.default_rng(42)
n, d = 10_000, 5
X = rng.normal(size=(n, d))      # (n, d)
w = rng.normal(size=d)           # (d,)
y = X @ w + rng.normal(scale=0.1, size=n)
```

CINCO PASSOS, SEMPRE NA MESMA ORDEM

1. Produto interno
2. Todas as previsões
3. Distâncias entre pares de pontos
4. MSE e gradiente
5. Conferir o gradiente e cronometrar

Em cada passo: laço, versão vetorizada, `np.allclose` entre as duas, `%timeit` nas duas.

1 Produto interno

COM LAÇO

```
x = X[0]                  # (5,)
total = 0.0
for j in range(d):
    total += x[j] * w[j]
```

VETORIZADO

```
total_v = X[0] @ w        # ()
np.isclose(total, total_v)   # True
```

SHAPES

`(5,) @ (5,) → ()`

As dimensões internas são iguais e somem: sobra um número.

ANOTE NO SEU NOTEBOOK

LAÇO	VETORIZADO	RAZÃO
_____ ms	_____ ms	_____ ×

2 Todas as previsões

COM LAÇO

```
y_hat = np.zeros(n)            # (n,)
for i in range(n):
    for j in range(d):
        y_hat[i] += X[i, j] * w[j]
```

VETORIZADO

```
y_hat_v = X @ w                # (n,)
np.allclose(y_hat, y_hat_v)    # True
%timeit X @ w
```

SHAPES

$(n, d) \; @ \; (d,) \rightarrow (n,)$

(10 000, 5) @ (5,) → (10 000,): uma previsão por amostra.

ANOTE NO SEU NOTEBOOK

LAÇO	VETORIZADO	RAZÃO
_____ ms	_____ ms	_____ ×

3 Distâncias entre pares de pontos

COM LAÇO

```
A, B = X[:200], X[:50]      # (200, 5), (50, 5)
D = np.zeros((200, 50))
for i in range(200):
    for j in range(50):
        D[i, j] = np.linalg.norm(A[i] - B[j])
```

VETORIZADO

```
# (200, 1, 5) - (1, 50, 5)
diff = A[:, None, :] - B[None, :, :]
# (200, 50, 5) → (200, 50)
D_v = np.sqrt((diff ** 2).sum(axis=2))
np.allclose(D, D_v)         # True
```

O `None` cria um eixo de tamanho 1 e o broadcasting faz todas as combinações. Para bases grandes, veja o truque do slide A5.

4 MSE e gradiente

COM LAÇO

```
w0 = np.zeros(d)
mse, grad = 0.0, np.zeros(d)
for i in range(n):
    erro = X[i] @ w0 - y[i]
    mse += erro ** 2 / n
    for j in range(d):
        grad[j] += 2 * erro * X[i, j] / n
```

VETORIZADO

```
erro = X @ w0 - y            # (n,)
mse_v = np.mean(erro ** 2)   # ()
grad_v = 2 / n * X.T @ erro  # (d,)
np.allclose(grad, grad_v)    # True
```

A terceira linha é a fórmula do quadro: $\nabla L = (2/n)\,\mathbf{X}^{\mathrm{T}}(\mathbf{X}\,\mathbf{w} - \mathbf{y})$.

5 Conferir o gradiente e cronometrar

```
def mse_de(w):
    return np.mean((X @ w - y) ** 2)

def grad_numerico(f, w, h=1e-6):
    g = np.zeros_like(w)
    for j in range(len(w)):
        e = np.zeros_like(w)
        e[j] = h
        g[j] = (f(w + e) - f(w - e)) / (2 * h)
    return g

np.allclose(grad_v, grad_numerico(mse_de, w0))  # True
```

A diferença central do Nível 3, aplicada a cada coordenada de $\mathbf{w}$.

Este laço percorre as d coordenadas, não as n amostras. É o único permitido no desafio.

Para fechar: rode `%timeit` nas versões dos passos 2 a 4 e anote a razão entre os tempos.

Erros comuns

* no lugar de @

`X * w` multiplica elemento a elemento e devolve (n, d), sem erro nenhum.

(n,) contra (n, 1)

`y.reshape(-1, 1) - y_hat` vira uma matriz (n, n) em silêncio. Cheque o shape antes da média.

axis trocado

`X.mean(axis=0)` é a média por feature; `axis=1` é por amostra.

Esquecer keepdims

`X.sum(axis=1)` tem shape (n,). Para subtrair por linha, use `keepdims=True` → (n, 1).

View alterando o original

Alterar uma fatia altera o array de origem. Use `.copy()` quando for modificar.

Inversa para resolver sistema

`np.linalg.inv` é instável. Use `solve` ou `lstsq`; o porquê está no slide A3.

Broadcasting que estoura a memória

(10 000, 1, 50) − (1, 10 000, 50) cria 5 bilhões de floats ≈ 40 GB. Use o truque do slide A5.

h pequeno demais

Com `h=1e-15` o arredondamento domina. Use diferença central com h entre 1e-4 e 1e-6.

Desafio: o módulo algebra.py

IMPLEMENTAR, SÓ COM NUMPY

`norma(x, ord=2)`
normas L1 e L2

`distancias(A, B)`
matriz (m, k) de distâncias euclidianas

`mse(X, y, w)` · `grad_mse(X, y, w)`
erro e gradiente na forma matricial

`grad_numerico(f, w, h=1e-6)`
diferença central por coordenada

CRITÉRIO DE ACEITE

- ✓ Nenhum `for` ou `while` fora de `grad_numerico`.
- ✓ `norma` bate com `np.linalg.norm` e `distancias` bate com `scipy.spatial.distance.cdist`, por `np.allclose`.
- ✓ `grad_mse` e `grad_numerico` concordam em cinco sementes diferentes.
- ✓ Tudo roda em menos de 1 segundo com X de shape (10 000, 50).

Correção no início da aula 6.2.

Leitura: documentação oficial do NumPy

NumPy: the absolute basics for beginners	numpy.org/doc/stable/user/absolute_beginners.html
Broadcasting	numpy.org/doc/stable/user/basics.broadcasting.html
numpy.matmul	numpy.org/doc/stable/reference/generated/numpy.matmul.html
Linear algebra (numpy.linalg)	numpy.org/doc/stable/reference/routines.linalg.html
numpy.einsum (para o slide A4)	numpy.org/doc/stable/reference/generated/numpy.einsum.html

Hessiana e convexidade

APROFUNDAMENTO · A1

MATRIZ DAS SEGUNDAS DERIVADAS

$$\mathbf{H} = \begin{vmatrix} \partial^2 f/\partial w_1^2 & \partial^2 f/\partial w_1 \partial w_2 \\ \partial^2 f/\partial w_2 \partial w_1 & \partial^2 f/\partial w_2^2 \end{vmatrix}$$

Para $f = w_1^2 + 3w_2^2$:

$$\mathbf{H} = \begin{vmatrix} 2 & 0 \\ 0 & 6 \end{vmatrix}$$

HESSIANA DO MSE

$$\mathbf{H} = (2/n)\,\mathbf{X}^T\mathbf{X}$$

$\mathbf{v}^T\mathbf{X}^T\mathbf{X}\,\mathbf{v} = \|\mathbf{X}\,\mathbf{v}\|^2 \geq 0$ para todo $\mathbf{v}$

→ $\mathbf{X}^T\mathbf{X}$ é semidefinida positiva

→ o MSE é convexo

→ **não existe mínimo local "falso"**

```
Hm = 2 / n * X.T @ X        # (d, d)
np.linalg.eigvalsh(Hm)      # todos ≥ 0
```

Autovalores e autovetores

APROFUNDAMENTO · A2

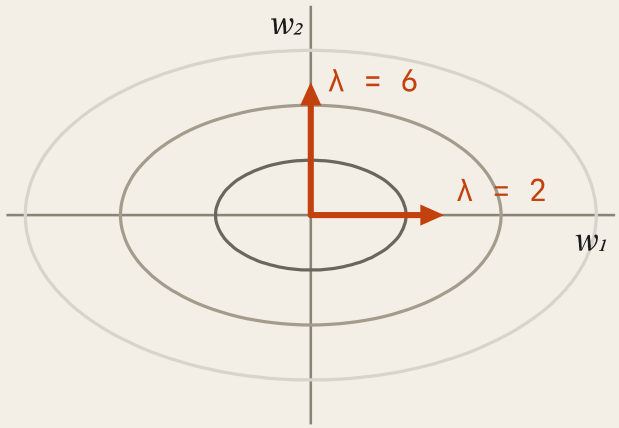
$$\mathbf{A}\,\mathbf{v} = \lambda\,\mathbf{v}$$

Direções que a matriz só estica, sem girar.

```
H = [[2, 0], [0, 6]]
λ = 2                    [1, 0]   eixo w₁
λ = 6                    [0, 1]   eixo w₂
```

```python
H = np.array([[2., 0.], [0., 6.]])
vals, vecs = np.linalg.eigh(H)
vals    # [2., 6.]
vecs    # colunas: [1, 0] e [0, 1]
```

Autovetores da matriz de covariância = componentes principais (PCA, aula 6.11).



Os eixos das elipses são os autovetores.

Número de condição

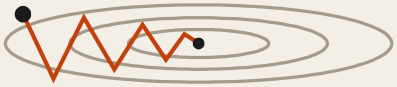
APROFUNDAMENTO · A3

$$\kappa = \lambda_{max} / \lambda_{min}$$

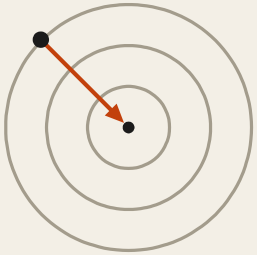
Para $\mathbf{H}$ = [[2, 0], [0, 6]]: κ = 6 / 2 = **3**.

κ grande: elipses alongadas, descida em zigue-zague (6.3) e inversa instável, o motivo do cartão “Inversa para resolver sistema”.

```
Xr = rng.normal(size=(1000, 2)) * [1, 100]
np.linalg.cond(Xr.T @ Xr)     # ≈ 1e4
Xs = (Xr - Xr.mean(0)) / Xr.std(0)
np.linalg.cond(Xs.T @ Xs)     # perto de 1
```



K GRANDE · SEM ESCALONAR



K ≈ 1 · ESCALONADO (5.4)

einsum: uma notação para todos os produtos

APROFUNDAMENTO · A4

Índice repetido que não aparece na saída é somado.

EINSUM	EQUIVALENTE	SHAPE
`np.einsum('j,j→', X[0], w)`	`X[0] @ w`	`()`
`np.einsum('ij,j→i', X, w)`	`X @ w`	`(n,)`
`np.einsum('ij,ij→i', X, X)`	`(X * X).sum(axis=1)`	`(n,)`
`np.einsum('ij,kj→ik', X[:3], X[:4])`	`X[:3] @ X[:4].T`	`(3, 4)`

Em `'ij,j→i'`: `j` aparece nas duas entradas e some da saída, então é somado; `i` sobrevive e vira o eixo do resultado.

Distâncias sem estourar a memória

APROFUNDAMENTO · A5

$$\|\mathbf{a}-\mathbf{b}\|^2 = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\,\mathbf{a}\cdot\mathbf{b}$$

```
A, B = X[:200], X[:50]                  # (200, 5), (50, 5)
aa = (A ** 2).sum(axis=1)[:, None]      # (200, 1)
bb = (B ** 2).sum(axis=1)[None, :]      # (1, 50)
D2 = aa + bb - 2 * A @ B.T              # (200, 50)
D = np.sqrt(np.maximum(D2, 0))          # (200, 50)
np.allclose(D, D_v)                     # True (passo 3)
```

MEMÓRIA INTERMEDIÁRIA

broadcasting	`(m, k, d)`
este truque	`(m, k)`

10 000 × 10 000 × 50	≈ 40 GB
10 000 × 10 000	≈ 0,8 GB

`np.maximum` evita raiz de –1e–16 por arredondamento. É o caminho para o desafio rodar em menos de 1 s.

Checagem de gradiente como profissional

APROFUNDAMENTO · A6

$$\text{erro relativo} = \|g - g_{num}\| \,/\, (\|g\| + \|g_{num}\|)$$

```
def erro_relativo(g, g_num):
    num = np.linalg.norm(g - g_num)
    den = np.linalg.norm(g) + np.linalg.norm(g_num)
    return num / den

erro_relativo(grad_v, grad_numerico(mse_de, w0))
```

< 1e-7	ótimo
1e-7 a 1e-4	suspeito: investigar
> 1e-4	há bug

Cheque em float64, o padrão do NumPy. float32 tem só cerca de 7 dígitos significativos.

PRÓXIMA AULA

6.2

Regressão linear pela equação normal

HOJE

$$\nabla L = (2/n)\,\mathbf{X}^{\mathrm{T}}(\mathbf{X}\,\mathbf{w} - \mathbf{y})$$

NA 6.2: IGUALAR A ZERO

$$\mathbf{X}^{\mathrm{T}}\mathbf{X}\,\mathbf{w} = \mathbf{X}^{\mathrm{T}}\mathbf{y}$$